

Image Recognition



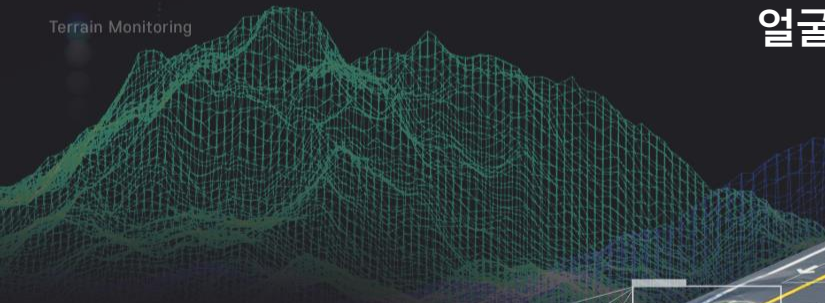
강원혁신플랫폼

컴퓨터 비전



얼굴 검출(DNN) 이미지

Terrain Monitoring



Autonomous Driving

Movement Analysis

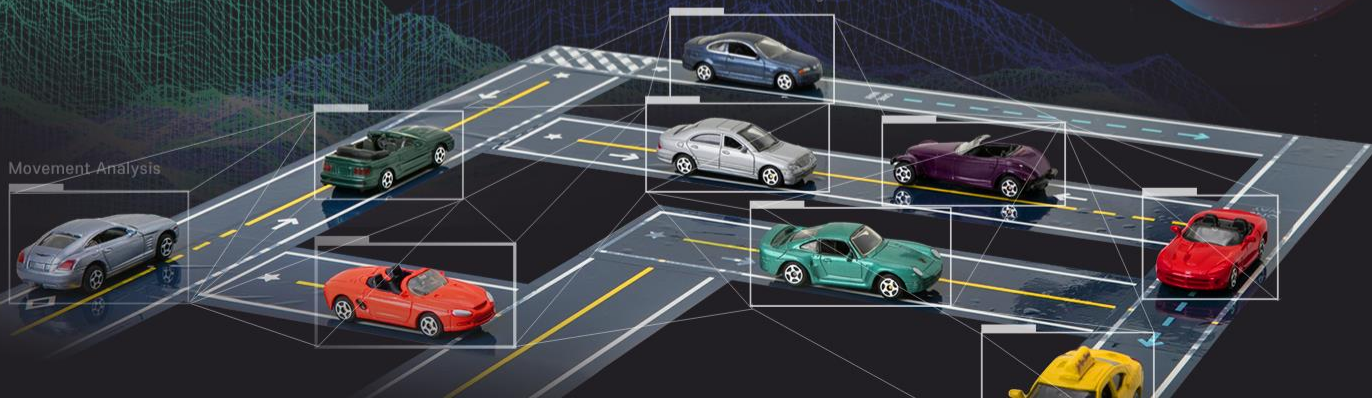
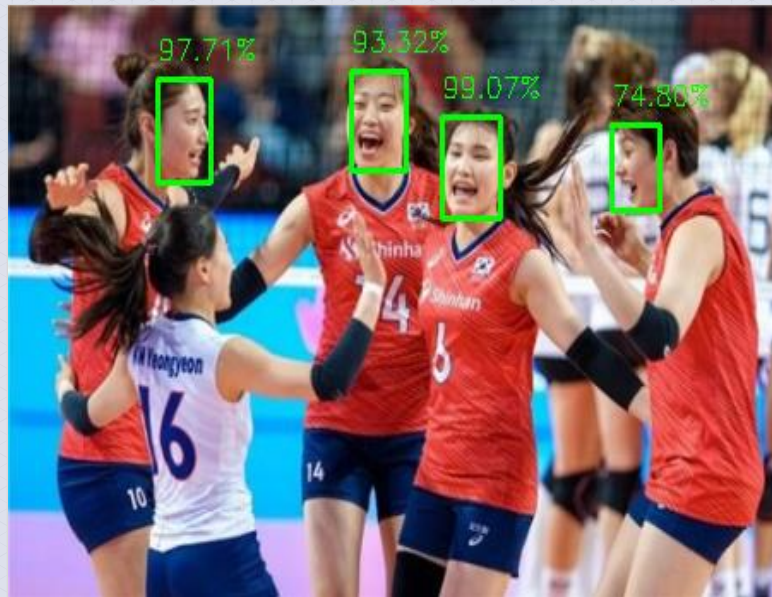
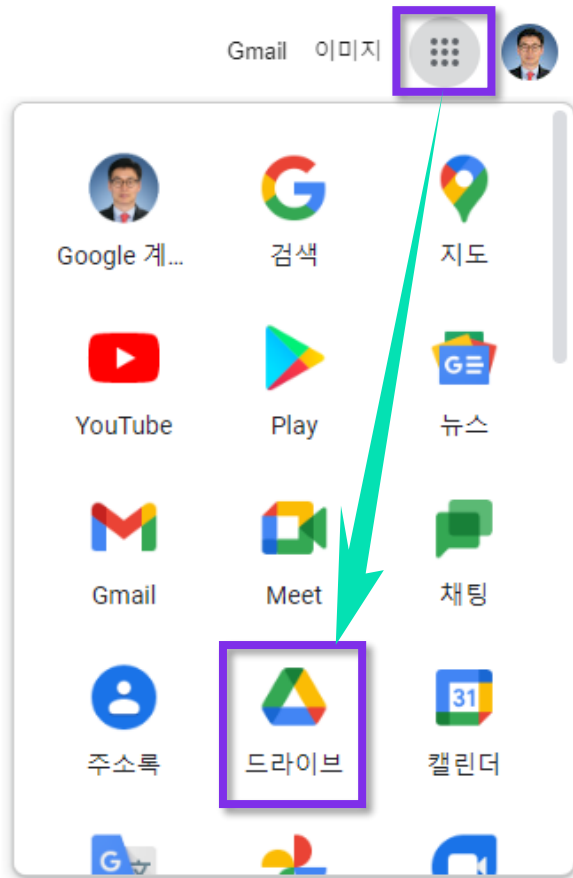


Image Face Detection

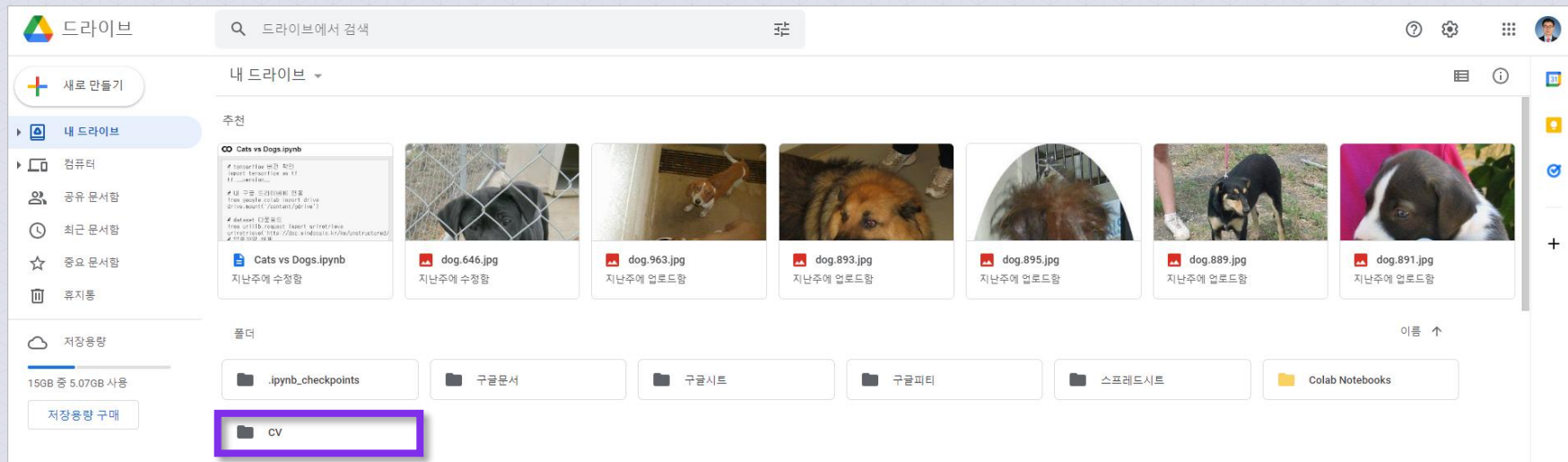
- OpenCV DNN Module 객체 감지 방식은 Deep Neural Network(DNN)으로 학습된 모델을 사용하여 이미지에서 얼굴을 감지함



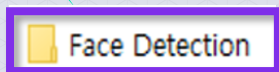
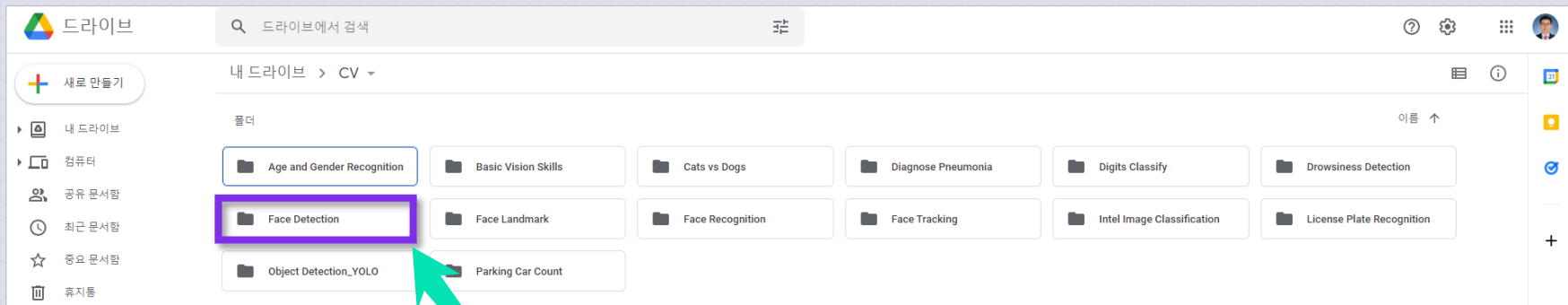


- 구글 Colab을 이용하기 위해
구글 크롬 브라우저에서 그림과 같이
구글 드라이브를 클릭한다.

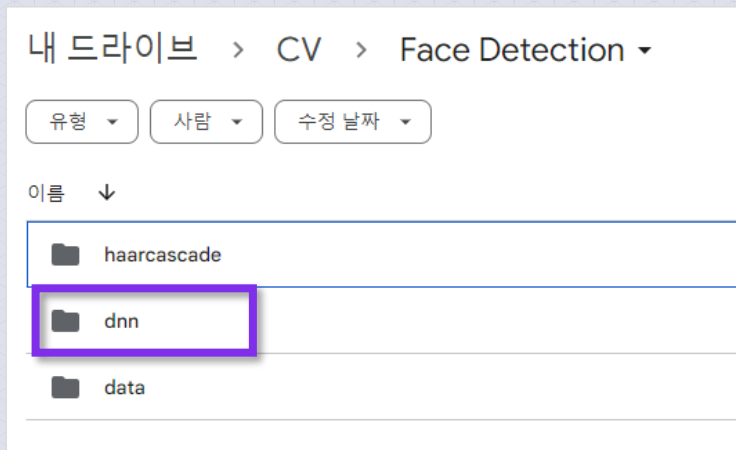
구글 드라이브에서 CV 폴더를 클릭한다.



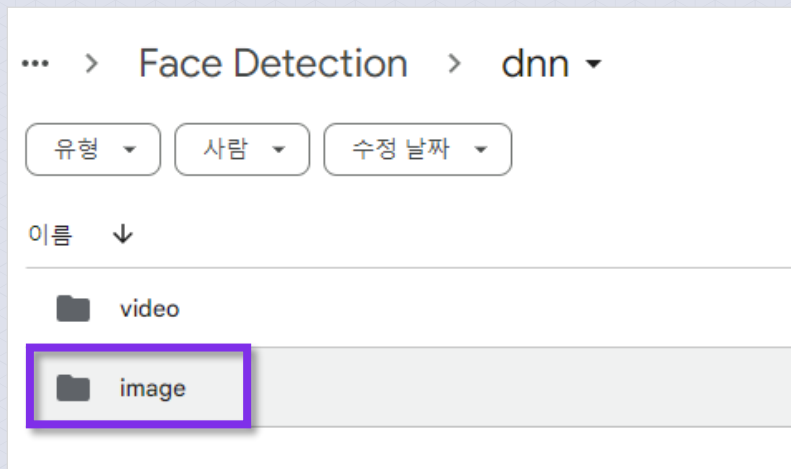
- 구글 드라이브 > CV > Face Detection 폴더를 클릭한다.
- 강의 실습 자료를 다운로드하여 [Face Detection] 폴더를 구글 드라이브 CV 폴더 아래에 복사한다.



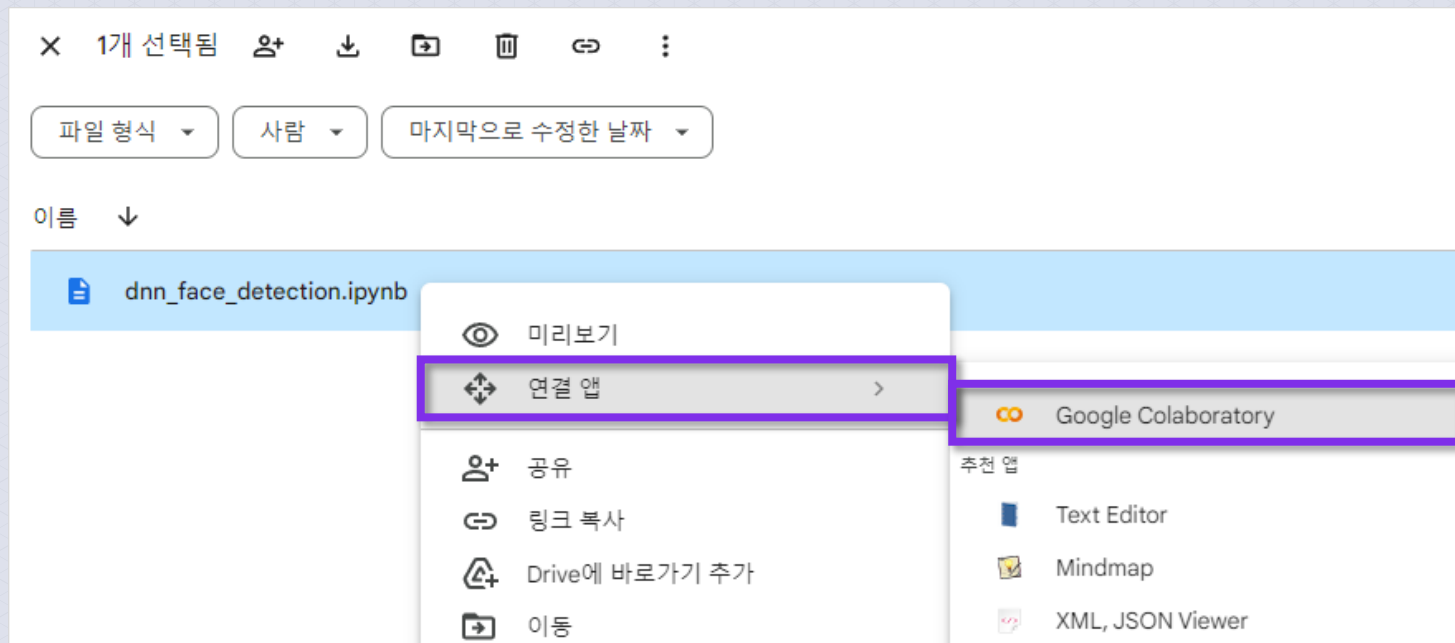
- ➡ 구글 드라이브 > CV > Face Detection > dnn 폴더를 클릭한다.
- ➡ Face Detection > dnn 폴더를 클릭한다.



- ➡ 구글 드라이브 > CV > Face Detection > dnn > image 폴더를 클릭한다.
- ➡ Face Detection > dnn > image 폴더를 클릭한다.



- ❶ 아래의 그림의 image 폴더에서 dnn_face_detection.ipynb를 선택 후 오른쪽 마우스를 클릭하여 “연결 앱 > Google Colaboratory”을 클릭한다.



↺ 아래 그림과 같이 구글 Colab 노트북 화면이 보이면 실습 준비가 완료된 것이다.



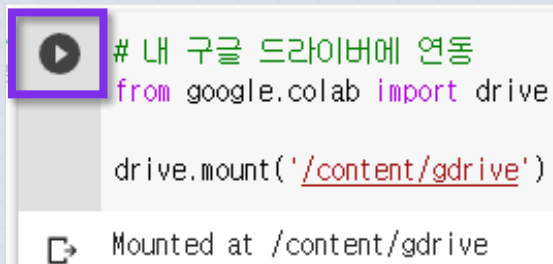
- ❶ 구글 Colab 노트북 화면에서 실행 버튼을 클릭함
- ❷ 실행 결과 TensorFlow 버전이 '2.8.2'인 것을 알 수 있음



```
# tensorflow 버전 확인  
import tensorflow as tf  
tf.__version__
```

```
'2.8.2'
```

- 구글 Colab 노트북 화면에서 실행 버튼을 클릭함



🔄 [Google Drive에 연결] > [계정 선택] > [구글 드라이버 접근 허용] 메뉴 선택

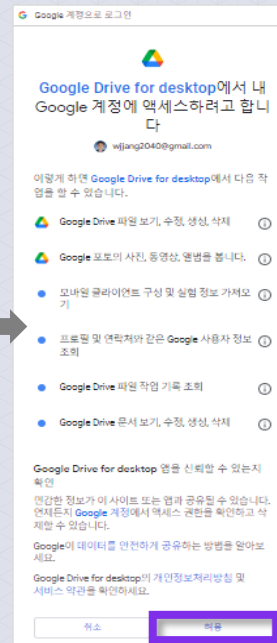
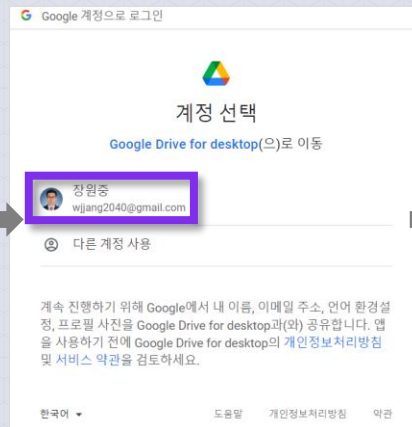
아래와 같이 Google Drive에 연동

노트북에서 Google Drive 파일에 액세스하도록 허용하시겠습니까?

이 노트북에서 Google Drive 파일에 대한 액세스를 요청합니다. Google Drive에 대한 액세스 권한을 부여하면 노트북에서 실행되는 코드가 Google Drive의 파일을 수정할 수 있게 됩니다. 이 액세스를 허용하기 전에 노트북 코드를 검토하시기 바랍니다.

아니요

Google Drive에 연결



구글 Colab 셀에 !git clone [GitHub web URL](#)로부터 다운로드함

- ➡ GitHub로부터 dataset을 다운로드함
- ➡ VM에 computer_vision라는 폴더가 생성됨
- ➡ computer_vision/CAFFE_DNN 폴더에서 필요한 파일만 나의 구글 드라이브로 복사함
- ➡ deploy.prototxt.txt, res10_300x300_ssd_iter_140000.caffemodel 파일만 복사함

▶ # Colab에서 GitHub 에 있는 데이터 가져오기
 # caffemodel 다운로드 URL : https://github.com/thegopieffect/computer_vision
 # Clone Web URL 를 복사해 옵니다.

```
!git clone https://github.com/thegopieffect/computer\_vision
```

➡ Cloning into 'computer_vision'...
 remote: Enumerating objects: 13, done.
 remote: Counting objects: 100% (6/6), done.
 remote: Compressing objects: 100% (4/4), done.
 remote: Total 13 (delta 0), reused 6 (delta 0), pack-reused 7
 Unpacking objects: 100% (13/13), done.

VM에 **computer_vision** 폴더를 확인함

- ➡ VM에 computer_vision 폴더가 생성되었는지 확인함

▶ # 구글 드라이브에 computer_vision 폴더 확인

```
!ls
```

📄 computer_vision gdrive sample_data

computer_vision/CAFF_DNN 폴더를 내 구글 드라이브로 복사 후
computer_vision 폴더를 제거함

- ➡ computer_vision/CAFF_DNN/deploy.prototxt.txt 파일을 복사함
- ➡ computer_vision/CAFF_DNN/res10_300x300_ssd_iter_140000.caffemodel 파일을 복사함



파일 복사

```
!cp computer_vision/CAFFE_DNN/deploy.prototxt.txt gdrive/My Drive/CV/Face Detection/data/  
!cp computer_vision/CAFFE_DNN/res10_300x300_ssd_iter_140000.caffemodel gdrive/My Drive/CV/Face Detection/data/  
print('files copy complete!!')
```



files copy complete!!

computer_vision/CAFF_DNN 폴더를 내 구글 드라이브로 복사 후
computer_vision 폴더를 제거함

🔄 VM의 computer_vision 폴더를 제거함

▶ # 다운로드 받았던 파일 제거(선택 사항)

```
!rm -r computer_vision
```

[] # 구글 드라이버에 폴더 확인

```
!!ls
```

```
gdrive sample_data
```

- 🔄 구글 Colab 노트북 화면에서 실행 버튼을 클릭함



필요한 패키지와 모듈 불러옴

```
import cv2  
from google.colab.patches import import cv2_imshow  
import numpy as np
```

- 구글 Colab 노트북 화면에서 실행 버튼을 클릭함

```
# OpenCV 버전 확인  
  
print("OpenCV version:")  
print(cv2.__version__)
```

```
OpenCV version:  
4.6.0
```

DNN으로 학습된 모델(Caffemodel)을 사용할 것을 정의함(학습된 모델 위치 정의)

- 학습된 DNN 모델의 물리적 파일 이름의 위치를 정의함
- 카페(Caffe) 딥러닝 프레임워크의 모델 파일 확장자는 .caffemodel임

```
# DNN으로 학습한 모델(caffemodel)을 사용
model_name = "gdrive/My Drive/CV/Face Detection/data/res10_300x300_ssd_iter_140000.caffemodel"
```


Model(Caffemodel)의 네트워크 구성파일을 정의함

- ➡ 모델 아키텍처가 정의된 파일 이름의 물리적 위치를 정의함
- ➡ 카페(Caffe) 딥러닝 프레임워크의 Config 파일 확장자는 .prototxt임



model Architecture 의 설계도 파일 위치 정의

```
prototxt_name = "gdrive/My Drive/CV/Face Detection/data/deploy.prototxt.txt"
```

얼굴 검출으로 인정할 최소 확률(신뢰도)과 이미지 넓이를 설정함

- 신뢰도가 40% 보다 큰 경우만 검출함
- 이미지 넓이를 450픽셀로 설정함

```
1 # detection 으로 인정할 최소 확률(신뢰도)
2 # - 여기서는 40% 보다 큰 경우 만 검출
3 min_confidence = 0.4
4
5 # 이미지 넓이를 설정
6 frame_width = 450
```

```
def detectAndDisplay(frame):
    # frame_width 에 맞춰 Image resize
    (height, width) = frame.shape[:2]
    ratio = frame_width / width
    dimension = (frame_width, int(height * ratio))
    frame = cv2.resize(frame, dimension, interpolation = cv2.INTER_AREA)

    # caffe모델의 weight 값과 모델 네트워크 구성을 불러와서 모델을 정의한다.
    model = cv2.dnn.readNetFromCaffe(prototxt_name, model_name)

    # 이미지를 300x300 으로 size를 조정하고 blob을 만든다.
    blob = cv2.dnn.blobFromImage(cv2.resize(frame, (300, 300)), 1.0, (300, 300), (104.0, 177.0, 123.0))

    # blob을 모델에 넣는다.
    model.setInput(blob)

    # detection을 수행한다.
    detections = model.forward()

    # detections 한 수만큼 루프가 돈다.
    for i in range(0, detections.shape[2]):

        confidence = detections[0, 0, i, 2] # confidence 는 detection한 확률을 나타냄

        # min_confidence 보다 큰 경우에만 detection으로 인정함
        if confidence > min_confidence:
            # detection 된 영역을 boxing
            # 상대적 좌표 + np.array([width, height, width, height]) 절대적인 boxing 좌표를 구해낸다
            # box = detections[0, 0, i, 3:7] + np.array([width, height, width, height])
            box = detections[0, 0, i, 3:7] * np.array([frame_width, int(height * ratio), frame_width, int(height * ratio)])
            (startX, startY, endX, endY) = box.astype("int")
            print(confidence, startX, startY, endX, endY)

            # 얼굴에 bounding box(사각형)를 그리고 확률값도 함께 나타낸다
            text = "{:.2f}%".format(confidence * 100)
            y = startY - 10 if startY - 10 > 10 else startY + 10
            cv2.rectangle(frame, (startX, startY), (endX, endY), (0, 255, 0), 2)
            cv2.putText(frame, text, (startX, y), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 1)

    # show the output image
    print("#### Face detection(DNN_CAFFE) ####")

    # 이미지를 Display 한다.
    cv2.imshow(frame)
```

원본 이미지에서
얼굴 검출하고 출력하는
detectAndDisplay() 함수를
정의함

(참고) OpenCV DNN(Deep Neural Network) 모듈

- 미리 학습된 딥러닝 파일을 OpenCV DNN 모듈로 실행할 수 있다.
- 순전파(Forward), 추론(Inference)만 가능하며 학습은 지원하지 않는다.
- OpenCV 3.3버전부터 기본 기능으로 제공한다.
- OpenCV 4.3버전부터 GPU(CUDA) 지원한다(단, 소스 코드 직접 빌드 필요).
- 지원하는 딥러닝 프레임워크 다음과 같다.



■ (참고) OpenCV DNN 모듈의 readNet() : 네트워크 불러오기

```
cv2.dnn.readNet(model, config=None, framework=None) # -> retval
```

model

훈련된 가중치를 저장하고 있는 이진 파일 이름

config

네트워크 구성을 저장하고 있는 텍스트 파일 이름

framework

명시적인 딥러닝 프레임워크 이름

retval

cv2.dnn_Net 클래스 객체

■ (참고) 딥러닝 프레임워크의 파일(Model, Config, Framework) 확장자

딥러닝 프레임워크	Model 파일 확장자	Config 파일 확장자	Framework 파일 확장자
Caffe	*.caffemodel	*.prototxt	"caffe"
TensorFlow	*.pb	*.pbtxt	"tensorflow"
Torch	*.t7 또는 *.net		"torch"
Darknet	*.weights	*.cfg	"darknet"
DLDT	*.bin	*.xml	"dldt"
ONNX	*.onnx		"onnx"

- (참고) OpenCV DNN 모듈의 blobFromImage() : 네트워크 입력 블록(blob) 만들기, 영상의 전처리 수행함

```
cv2.dnn.blobFromImage(image, scalefactor=None, size=None, mean=None, swapRB=None, crop=None, ddepth=None) -> retval
```

image

입력 영상

scalefactor

입력 영상 픽셀 값에 곱할 값, 기본 값 : 1

size

출력 영상의 크기, 기본 값 : (0, 0)

mean

입력 영상 각 채널에서 뺄 평균 값, 기본 값 : (0, 0, 0, 0)

swapRB

R과 B 채널을 서로 바꿀 것인지를 결정하는 플래그,
기본 값 : False

crop

크롭(Crop) 수행 여부, 기본 값 : False

ddepth

출력 블록의 깊이, CV_32F 또는 CV_8U, 기본 값 : CV_32F

retval

영상으로부터 구한 블록 객체

`numpy.ndarray.shape=(N,C,H,W).dtype=numpy.float32.`

■ (참고) OpenCV DNN 모듈의 setInput() : 네트워크 입력 설정하기

```
cv2.dnn_Net.setInput(blob, name=None, scalefactor=None, mean=None) # -> None
```

blob

블롭 객체

name

입력 레이어 이름

scalefactor

추가적으로 픽셀 값에 곱할 값

mean

추가적으로 픽셀 값에서 뺄 평균 값

■ (참고) OpenCV DNN 모듈의 forward() : 네트워크 순방향 실행(추론)

```
cv2.dnn_Net.forward(outputName=None) # -> retval  
cv2.dnn_Net.forward(outputNames=None, outputBlobs=None) # -> outputBlobs
```

outputName

출력 레이어 이름

retval

지정한 레이어의 출력 블록으로 네트워크마다 다르게 결정됨

outputNames

출력 레이어 이름 리스트

outputBlobs

지정한 레이어의 출력 블록 리스트

얼굴 검출 원본 이미지를 정의함

- 구글 Colab 노트북 화면에서 실행 버튼을 클릭함

```
1 # 얼굴 검출 원본 이미지
2 file_name1 = "gdrive/My Drive/CV/Face Detection/data/image/face_05.jpg"
3 file_name2 = "gdrive/My Drive/CV/Face Detection/data/image/face_06.jpg"
4 file_name3 = "gdrive/My Drive/CV/Face Detection/data/image/sports_01.jpg"
5 file_name4 = "gdrive/My Drive/CV/Face Detection/data/image/sports_02.jpg"
```



file_name1



file_name2



file_name3



file_name4

원본 이미지(1)을 출력함

```
1  # 원본 이미지1 화면 출력
2  img1 = cv2.imread(file_name1)
3  print("shape: {}".format(img1.shape))          # shape: (854, 1280, 3)
4  print("width: {} pixels".format(img1.shape[1]))
5  print("height: {} pixels".format(img1.shape[0]))
6  print("channels: {}".format(img1.shape[2]))
7
8  cv2.imshow(img1)
```



원본 이미지(2)를 출력함

```
1  # 원본 이미지2 화면 출력
2  img2 = cv2.imread(file_name2)
3  print("shape: {}".format(img2.shape))          # shape: (853, 1280, 3)
4  print("width: {} pixels".format(img2.shape[1]))
5  print("height: {} pixels".format(img2.shape[0]))
6  print("channels: {}".format(img2.shape[2]))
7
8  cv2.imshow(img2)
```



원본 이미지(3)을 출력함

```
1 # 원본 이미지3 화면 출력
2 img3 = cv2.imread(file_name3)
3 print("shape: {}".format(img3.shape))           # shape: (349, 620, 3)
4 print("width: {} pixels".format(img3.shape[1]))
5 print("height: {} pixels".format(img3.shape[0]))
6 print("channels: {}".format(img3.shape[2]))
7
8 cv2.imshow(img3)
```



원본 이미지(4)를 출력함

```
1 # 원본 이미지4 화면 출력
2 img4 = cv2.imread(file_name4)
3 print("shape: {}".format(img4.shape))           # shape: (333, 500, 3)
4 print("width: {} pixels".format(img4.shape[1]))
5 print("height: {} pixels".format(img4.shape[0]))
6 print("channels: {}".format(img4.shape[2]))
7
8 cv2.imshow(img4)
```

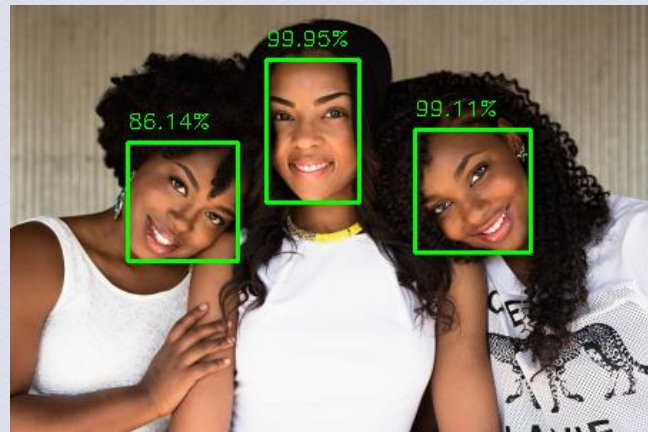


원본 이미지(1)에서 얼굴 검출하고 출력하는 함수를 실행함

아래 그림에서 얼굴이 정면이 아닌 돌려져 있어도 인식률이 높은 것을 알 수 있음

```
1 # 원본 이미지(1)에서 얼굴 검출하기
2 detectAndDisplay(img1)
3
4 #
5 # dnn module face detection은 얼굴이 정면이 아닌 돌려져 있어도 인식률이 높은 것을 볼 수 있다.
6 #
```

```
0.99945337 179 38 244 138
0.99113595 283 87 364 173
0.86141443 82 96 159 179
```

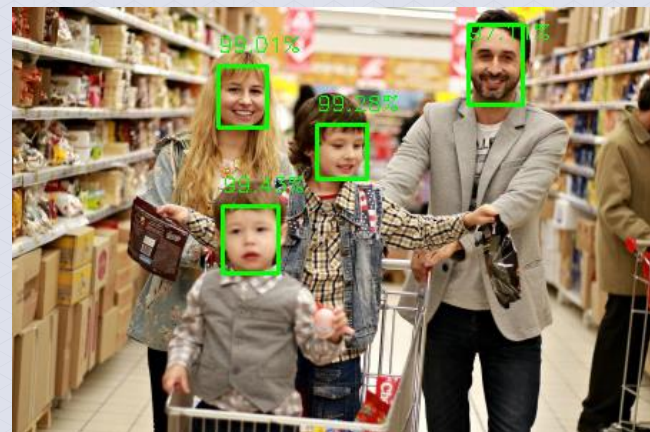


원본 이미지(2)에서 얼굴 검출하고 출력하는 함수를 실행함

아래 그림에서 얼굴이 높은 인식률로 검출된 것을 알 수 있음

```
1 # 원본 이미지(2)에서 얼굴 검출하기
2 detectAndDisplay(img2)
3
4 #
5 # dnn module face detection은 얼굴 잘 검출된 것을 볼 수 있다.
6 #
```

```
0.9943474 147 139 186 186
0.99281174 213 82 248 120
0.9901316 144 41 178 84
0.9711057 319 12 357 68
```



아래 그림에서 얼굴 이미지 크기가 작아져도 인식률이 높은 것을 알 수 있음

```

0.99945337 179 38 244 138
0.99113595 283 87 364 173
0.86141443 82 96 159 179

```

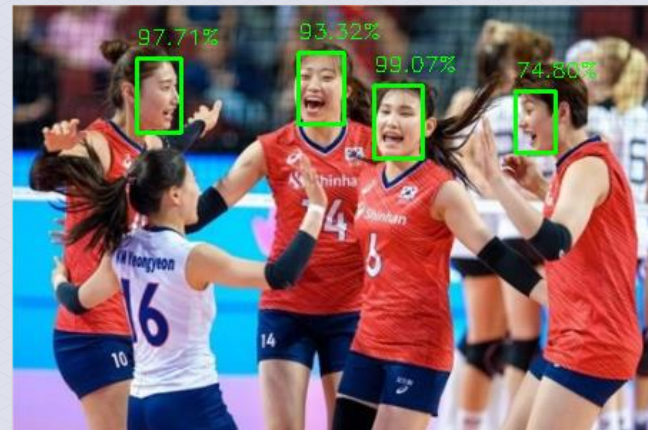


원본 이미지(4)에서 얼굴 검출하고 출력하는 함수를 실행함

아래 그림에서 얼굴이 정면이 아닌 돌려져 있어도 인식률이 높은 것을 알 수 있음

```
1 # 원본 이미지(4)에서 얼굴 검출하기
2 detectAndDisplay(img4)
3
4 #
5 # dnn module face detection은 얼굴이 정면이 아닌 돌려져 있어도 인식률이 높은 것을 볼 수 있다.
6 #
```

```
0.99067104 253 56 287 107
0.9770544 87 37 118 89
0.93315625 200 33 232 83
0.7479832 352 60 380 103
```





■ DNN Module Face Detection의 특성

1

얼굴 검출 속도는 느리나 정확도가 높음

2

얼굴이 정면이 아닌 돌려져 있어도 인식률이 높음

3

이미지의 크기(해상도)가 작아져도 인식률이 높음